

Факультет компьютерных наук

14 ноября 2026 г.

Полносвязные нейронные сети

Жарковский Дмитрий Андреевич
garkovski_dmitri@mail.ru

Простейшая задача классификации

По фотографии выбрать один из двух классов



кошка



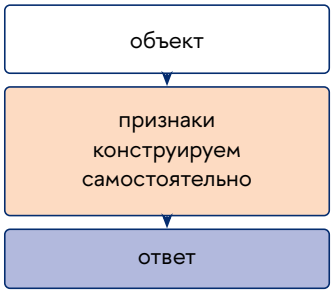
собака

- Классов два: кошка или собака.
- Какие признаки вы бы выделили для этой задачи?

Зачем нейронные сети

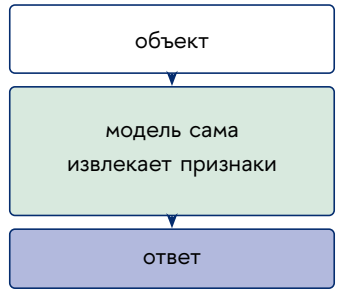
Кто извлекает признаки объекта

Классическое машинное обучение



- Формулу признака задаём и считаем сами.
- Модель видит готовую таблицу чисел.

Нейронная сеть



- Ручные формулы признаков не пишем.
- Полезные признаки находит обучение.



Линейная модель

Предсказание стоимости квартиры

- Ответ — сумма признаков, каждый со своим весом:

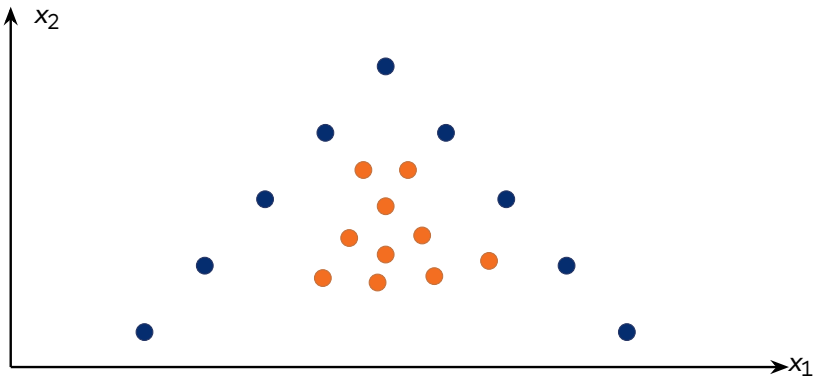
$$a(x) = w_0 + w_1 \cdot (\text{площадь}) + w_2 \cdot (\text{этаж})$$

$$+ w_3 \cdot (\text{расстояние до метро}) + \cdots$$

- Признаки, скорее всего, связаны между собой.
- Признак и целевая переменная могут быть нелинейно зависимы.
- Градиентный спуск хочется оставить, а модель — усложнить.

Исходные данные

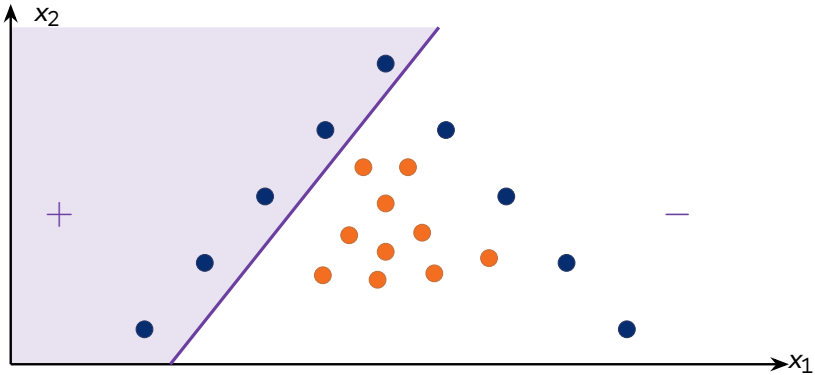
Классы нельзя разделить одной прямой



- Оранжевые точки находятся внутри, а синие окружают их снаружи: нужна нелинейная граница.

Первая линейная модель

Отделяем левую границу оранжевой области

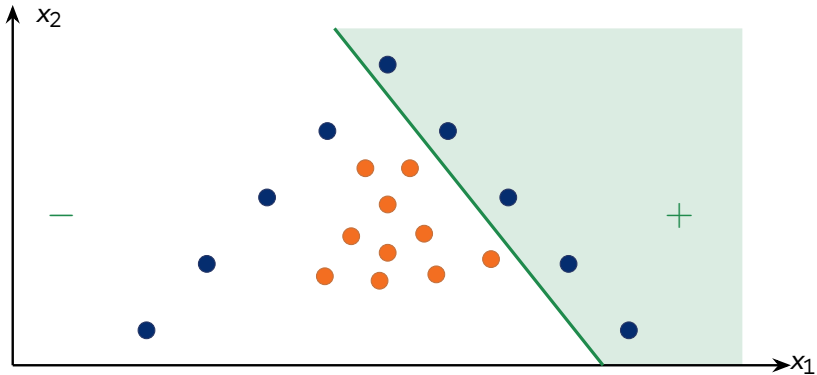


$$b_1(x) = sign(-2x_1 + x_2 + 5)$$

- Модель $b_1(x)$ делит плоскость на две полуплоскости и определяет знак каждой точки.

Вторая линейная модель

Отделяем правую границу оранжевой области

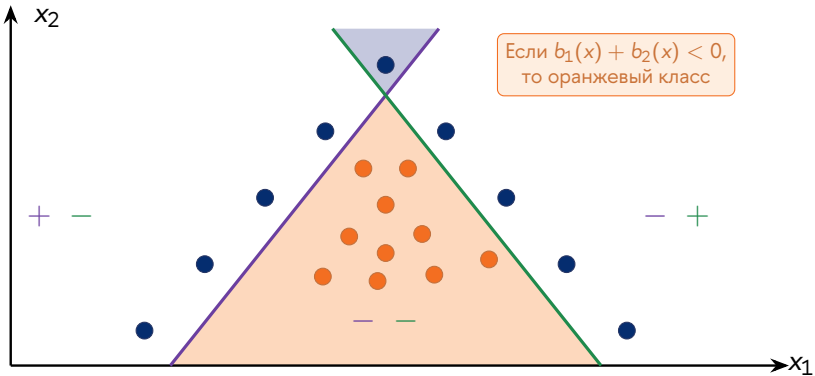


$$b_2(x) = sign(2x_1 + x_2 - 5)$$

- Модель $b_2(x)$ проводит вторую границу; вместе прямые выделяют центральную область.

Объединяем две модели

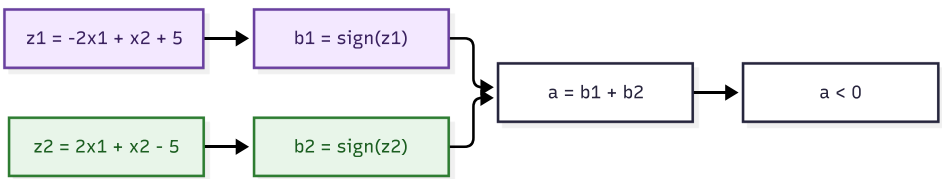
Комбинация линейных правил задаёт нелинейную область



- Если $b_1(x) + b_2(x) < 0$, точка попадает между прямыми и относится к оранжевому классу.

Композиция линейных моделей

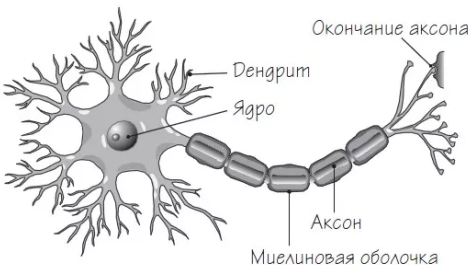
Две формулы, потом правило объединения



- Сначала две линейные формулы, затем их знаки, затем сумма.
- Последняя клетка проверяет, отрицательна ли сумма.

Настоящий нейрон

Много входов, один ответ



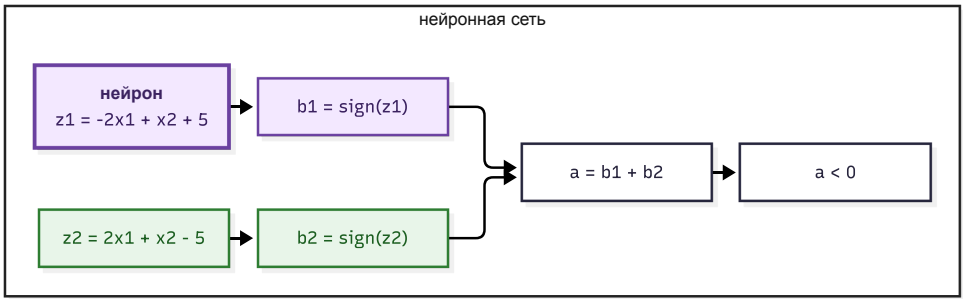
- Искусственный нейрон повторяет идею: много входов, один ответ.
- Ответ — взвешенная сумма входов:

$$a(x) = \sum_{j=1}^{d} w_j x_j.$$

- Вход умножаем на вес и все произведения складываем.

Нейрон и нейронная сеть

Один блок и весь граф вычислений



- Блок с линейной формулой — как бы нейрон.
- Весь граф — как бы нейронная сеть.



Слой как умножение на матрицу

Полносвязный слой

- Это умножение вектора на матрицу:

$$z = Wx + b.$$

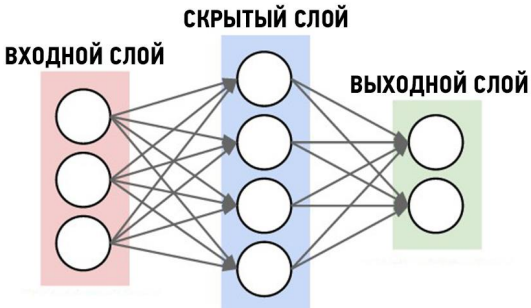
- Строка матрицы W — веса одного выхода. Пример, три входа и четыре выхода:

$$\overbrace{\begin{pmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{pmatrix}}^{z} = \overbrace{\begin{pmatrix} 1 & 2 & 0 \\ 0 & -1 & 1 \\ 3 & 1 & -2 \\ -1 & 0 & 2 \end{pmatrix}}^{W} \overbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}}^{x} + \overbrace{\begin{pmatrix} 1 \\ 0 \\ -2 \\ 1 \end{pmatrix}}^{b}.$$

- У каждого из четырёх выходов — своя строка весов и своё смещение.

Полносвязный слой

Каждый узел связан со всеми узлами следующего слоя



Сколько параметров?

$$N_{\text{параметров}} = (n_{\text{входов}} + 1) \cdot n_{\text{выходов}}$$

Линейные слои подряд

На простом примере два слоя превращаются в один

Первый слой

$$z = 2x + 1$$

Второй слой

$$a = 3z - 4$$

Подставим ответ первого слоя во второй:

$$a = 3(2x + 1) - 4$$

$$a = 6x - 1$$

Вывод

Два линейных слоя дают одну линейную модель.

Сигмоида

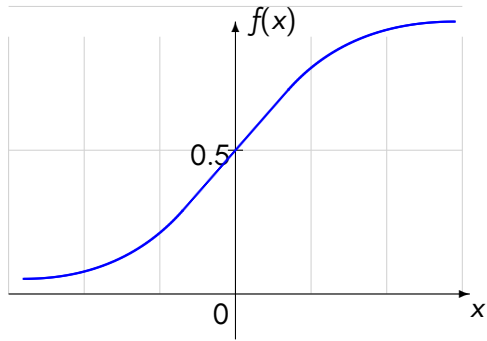
Нелинейность после суммы

- После взвешенной суммы ставим функцию f:

$$z_j = f\Big(\sum_{i=1}^{n} w_{ji} x_i + b_j\Big).$$

- Сигмоида сжимает любое число к диапазону от 0 до 1:

$$f(x) = \frac{1}{1 + \exp(-x)}.$$



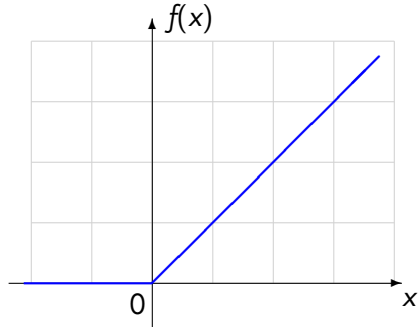
ReLU

Отрицательное обнуляем, положительное оставляем

- Та же схема: сумма, затем функция f.
- ReLU, rectified linear unit:

$$f(x) = \max(0, x).$$

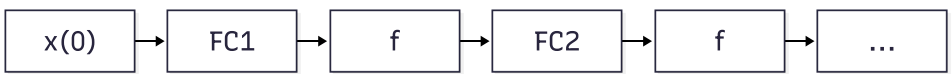
- Из-за излома в нуле стопка слоёв уже не равна одной прямой.





Типичная полносвязная сеть

Линейный слой и нелинейность чередуются

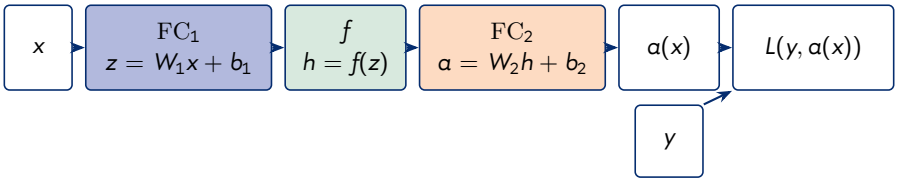


- $x^{(0)}$ — входные числа.
- FC — полносвязный слой, f — сигмоида или ReLU.
- Так можно повторять несколько раз.

Forward pass

Прямой проход: от входа к ответу

- Forward pass — вычисление ответа сети по заданному входу.
- Идём слева направо: выход блока становится входом следующего.
- Веса в формулах уже заданы и на этом шаге не меняются.



- Ответ сети — $a(x)$. Ошибка $L(y, a(x))$ сравнивает его с верным ответом y.
- Веса подбираем по средней ошибке на ℓ примерах:

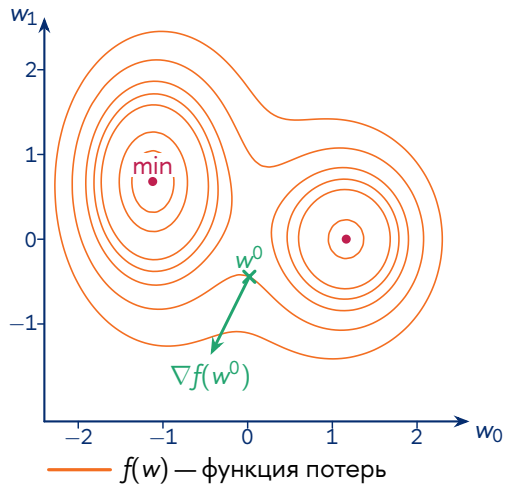
$$\frac{1}{\ell} \sum_{i=1}^{\ell} L(y_i, a(x_i)) \to \min_w$$

Градиентный спуск

Веса сдвигаем против градиента

- $w = (w_0, w_1)$.
- Градиент $\nabla f(w)$ направлен в сторону роста ошибки.
- Градиент функции потерь —

$$\nabla f(w) = \left(\frac{\partial f}{\partial w_0}, \frac{\partial f}{\partial w_1} \right).$$

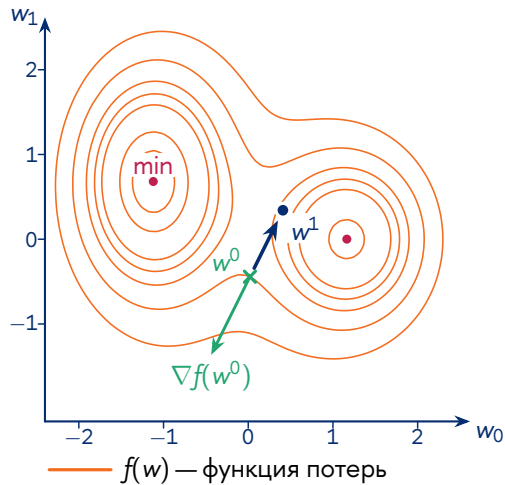


Градиентный спуск

Формула одного шага

$$w^1 = w^0 - \eta \nabla f(w^0)$$

- w^1 — веса после шага.
- w^0 — веса до шага.
- Минус — шаг в сторону убывания ошибки.
- η — скорость обучения: насколько сильно меняем веса.
- $\nabla f(w^0)$ — градиент ошибки в точке w^0.



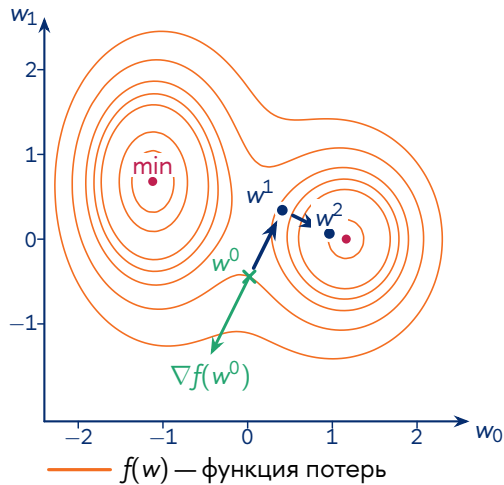
Градиентный спуск

Следующий шаг

- Следующий шаг из w^1:

$$w^2 = w^1 - \eta \nabla f(w^1).$$

- Не гарантируется, что мы придём в глобальный минимум.
- Шаг может уйти в другую впадину.



Производная ошибки по весу

Что нужно, чтобы сделать шаг

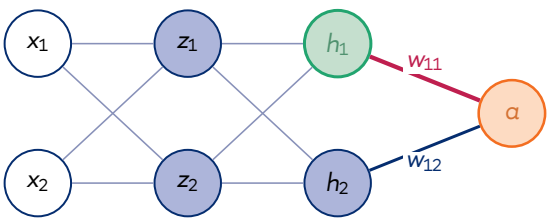
- Чтобы сдвинуть вес w_j, нужна производная ошибки $\partial L / \partial w_j$.
- Градиент $\nabla f(w)$ собран из таких производных по всем весам.
- Для MSE на одном примере $L = \big(a(x_i, w) - y_i\big)^2$.

$$\frac{\partial L}{\partial w_j} = \frac{\partial}{\partial w_j}\big(a(x_i, w) - y_i\big)^2 = \underbrace{2\big(a(x_i, w) - y_i\big)}_{\text{влияние ответа}} \cdot \underbrace{\frac{\partial}{\partial w_j} a(x_i, w)}_{\text{влияние веса}}.$$

- Влияние ответа: как функция потерь меняется при изменении ответа модели.
- Влияние веса: как ответ сети меняется при сдвиге w_j.

Производная ответа по весу

Последний слой, вес w_{11}



Линейный выход. h_1 и h_2 берём готовыми:

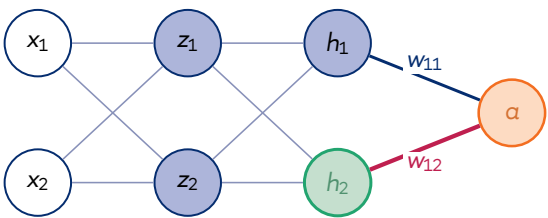
$$a = w_{11}\, h_1 + w_{12}\, h_2.$$

Вес w_{11} входит только в первое слагаемое:

$$\frac{\partial a}{\partial w_{11}} = h_1.$$

Производная ответа по весу

Последний слой, вес w_{12}



Линейный выход. h_1 и h_2 берём готовыми:

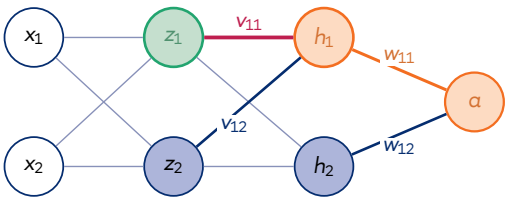
$$a = w_{11}\, h_1 + w_{12}\, h_2.$$

Вес w_{12} входит только во второе слагаемое:

$$\frac{\partial a}{\partial w_{12}} = h_2.$$

Производная ответа по весу

Цепочка от v_{11} до выхода a



Линейный нейрон. z_1 и z_2 уже посчитаны:

$$h_1 = v_{11}\, z_1 + v_{12}\, z_2.$$

До выхода a ведёт вес w_{11}:

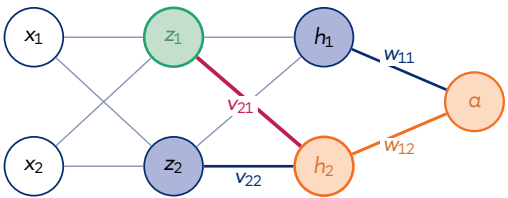
$$a = w_{11}\, h_1 + w_{12}\, h_2.$$

Вес v_{11} входит только в первое слагаемое h_1:

$$\frac{\partial a}{\partial v_{11}} = \frac{\partial a}{\partial h_1} \cdot \frac{\partial h_1}{\partial v_{11}} = w_{11} \cdot z_1.$$

Производная ответа по весу

Цепочка от v_{21} до выхода a



Линейный нейрон. z_1 и z_2 уже посчитаны:

$$h_2 = v_{21}\, z_1 + v_{22}\, z_2.$$

До выхода a ведёт вес w_{12}:

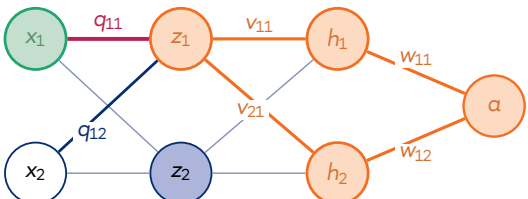
$$a = w_{11}\, h_1 + w_{12}\, h_2.$$

Вес v_{21} входит только в первое слагаемое h_2:

$$\frac{\partial a}{\partial v_{21}} = \frac{\partial a}{\partial h_2} \cdot \frac{\partial h_2}{\partial v_{21}} = w_{12} \cdot z_1.$$

Производная ответа по весу

Первый слой, вес q_{11}



Линейный нейрон. x_1 и x_2 берём готовыми:

$$z_1 = q_{11}\, x_1 + q_{12}\, x_2.$$

От z_1 к выходу ведут два пути:

$$h_1 = v_{11}\, z_1 + \cdots, \qquad h_2 = v_{21}\, z_1 + \cdots.$$

Вклады путей складываются. $\partial z_1 / \partial q_{11} = x_1$:

$$\frac{\partial a}{\partial q_{11}} = \left(\frac{\partial a}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial q_{11}} + \frac{\partial a}{\partial h_2} \cdot \frac{\partial h_2}{\partial z_1} \cdot \frac{\partial z_1}{\partial q_{11}} \right) = \left(w_{11}\, v_{11} + w_{12}\, v_{21} \right) \cdot x_1.$$

Backpropagation

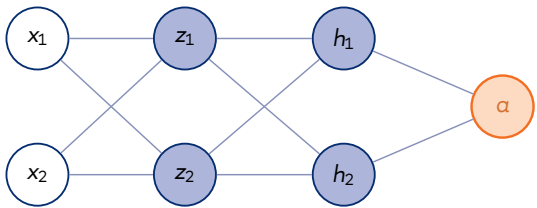
Обратный проход: производные считаем с конца

3: $\frac{\partial a}{\partial h_1} \quad \frac{\partial a}{\partial h_2}$

2: $\frac{\partial a}{\partial z_1} = \frac{\partial a}{\partial h_1}\frac{\partial h_1}{\partial z_1} + \frac{\partial a}{\partial h_2}\frac{\partial h_2}{\partial z_1} \quad \frac{\partial a}{\partial z_2} = \frac{\partial a}{\partial h_1}\frac{\partial h_1}{\partial z_2} + \frac{\partial a}{\partial h_2}\frac{\partial h_2}{\partial z_2}$

1: $\frac{\partial a}{\partial x_1} = \frac{\partial a}{\partial h_1}\frac{\partial h_1}{\partial z_1}\frac{\partial z_1}{\partial x_1} + \frac{\partial a}{\partial h_2}\frac{\partial h_2}{\partial z_1}\frac{\partial z_1}{\partial x_1} + \frac{\partial a}{\partial h_1}\frac{\partial h_1}{\partial z_2}\frac{\partial z_2}{\partial x_1} + \frac{\partial a}{\partial h_2}\frac{\partial h_2}{\partial z_2}\frac{\partial z_2}{\partial x_1}$

$\frac{\partial a}{\partial x_2} = \frac{\partial a}{\partial h_1}\frac{\partial h_1}{\partial z_1}\frac{\partial z_1}{\partial x_2} + \frac{\partial a}{\partial h_2}\frac{\partial h_2}{\partial z_1}\frac{\partial z_1}{\partial x_2} + \frac{\partial a}{\partial h_1}\frac{\partial h_1}{\partial z_2}\frac{\partial z_2}{\partial x_2} + \frac{\partial a}{\partial h_2}\frac{\partial h_2}{\partial z_2}\frac{\partial z_2}{\partial x_2}$



Теорема Цыбенко

Теорема (Цыбенко)

Сеть с одним скрытым слоем и сигмоидой может приблизить любую непрерывную зависимость на ограниченной области сколь угодно точно, если нейронов достаточно.

- Теорема говорит, что такое приближение возможно.
- Она не говорит, как подобрать веса.

Поэкспериментируйте

Сеть можно собрать в браузере

- Посмотрите, как прямая, скрытый слой и ReLU меняют границу классов.
- TensorFlow Playground: https://playground.tensorflow.org

Резюме

- Полносвязный слой — умножение вектора входов на матрицу весов, плюс смещение.
- Композиция линейных слоев — линейный слой.
- Сеть с одним скрытым слоем и нелинейностью может приблизить любую непрерывную зависимость.